

Core Language Working Group "tentatively ready" Issues for the July, 2017 (Toronto) meeting

Section references in this document reflect the section numbering of document [WG21 N4659](#).

1523. Point of declaration in range-based for

Section: 9.5.4 [stmt.ranged] **Status:** tentatively ready **Submitter:** John Spicer **Date:** 2012-07-16

According to the general rule for declarations in 6.3.2 [basic.scope.pdecl] paragraph 1,

The *point of declaration* for a name is immediately after its complete declarator (Clause 11 [dcl.decl]) and before its *initializer* (if any), except as noted below.

However, the rewritten expansion of the range-based `for` statement in 9.5.4 [stmt.ranged] paragraph 1 contradicts this general rule, so that the index variable is not visible in the *range-init*:

```
for (int i : {i}) ;    // error: i not in scope
```

(See also issue 1498 for another question regarding the rewritten form of the range-based `for`.)

Notes from the October, 2012 meeting:

EWG is discussing issue 900 and the outcome of that discussion should be taken into consideration in addressing this issue.

Notes from the April, 2013 meeting:

The approach favored by CWG for resolving this issue is to change the point of declaration of the variable in the *for-range-declaration* to be after the `)`.

Proposed resolution (May, 2017):

Add the following as a new paragraph following 6.3.2 [basic.scope.pdecl] paragraph 9:

The point of declaration for a function-local predefined variable (11.4 [dcl.fct.def]) is immediately before the *function-body* of a function definition.

The point of declaration for the variable or the structured bindings declared in the *for-range-declaration* of a range-based `for` statement (9.5.4 [stmt.ranged]) is immediately after the *for-range-initializer*.

The point of declaration for a template parameter...

1704. Type checking in explicit instantiation of variable templates

Section: 17.7.2 [temp.explicit] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2013-06-20

It is not clear whether the following is well-formed or not:

```
template<typename T> int arr[sizeof(T)] = {};  
template int arr<int>[];
```

Are we supposed to instantiate the specialization and treat the explicit instantiation declaration as if it were a redeclaration (in which case the omitted array bound would presumably be OK), or is the type of the explicit instantiation declaration required to exactly match the type that the instantiated specialization has (in which case the omitted bound would presumably not be OK)? Or something else?

(See also issue 1728.)

Proposed resolution (May, 2017):

1. Change 17.7.2 [temp.explicit] paragraph 3 as follows:

If the explicit instantiation is for a class or member class, the *elaborated-type-specifier* in the *declaration* shall include a *simple-template-id*; **otherwise, the declaration shall be a simple-declaration whose *init-declarator-list* comprises a single *init-declarator* that does not have an *initializer*.** If the explicit instantiation is for a function or member function, the *unqualified-id* in the **declaration declarator** shall be either a *template-id* or, where all template arguments can be deduced, a *template-name* or *operator-function-id*. [Note: The declaration may declare a *qualified-id*, in which case the *unqualified-id* of the *qualified-id* must be a *template-id*. —end note] If the explicit instantiation is for a member

function, a member class or a static data member of a class template specialization, the name of the class template specialization in the *qualified-id* for the member name shall be a *simple-template-id*. If the explicit instantiation is for a variable **template specialization**, the *unqualified-id* in the **declaration declarator** shall be a *simple-template-id*. An explicit instantiation shall appear in an enclosing namespace of its template. If the name declared in the explicit instantiation is an unqualified name, the explicit instantiation shall appear in the namespace where its template is declared or, if that namespace is inline (10.3.1 [namespace.def]), any namespace from its enclosing namespace set. [Note:...

2. Add the following as a new paragraph following 17.7.2 [temp.explicit] paragraph 4:

The declaration in an explicit-instantiation and the declaration produced by the corresponding substitution into the templated function, variable, or class are two declarations of the same entity. [Note: These declarations are required to have matching types as specified in 6.5 [basic.link], except as specified in 18.4 [except.spec]. [Example:

```
template<typename T> T var = {};  
template float var<float>; // OK, instantiated variable has type float  
template int var<int[16]>[]; // OK, absence of major array bound is permitted  
template int *var<int>; // error: instantiated variable has type int  
  
template<typename T> auto av = T();  
template int av<int>; // OK, variable with type int can be redeclared with type auto  
  
template<typename T> auto f() {}  
template void f<int>(); // error: function with deduced return type redeclared with non-deduced return type (10.1.7.4 [dcl.spec.auto])
```

—end example] —end note] Despite its syntactic form, the declaration in an explicit-instantiation for a variable is not itself a definition and does not conflict with the definition instantiated by an explicit instantiation definition for that variable.

3. Change 17.7.2 [temp.explicit] paragraph 10 as follows:

Except for inline functions and variables, declarations with types deduced from their initializer or return value (10.1.7.4 [dcl.spec.auto]), const variables of literal types, variables of reference types, and class template specializations, explicit instantiation declarations have the effect of suppressing the implicit instantiation of the **definition of the** entity to which they refer. [Note:...

This resolution also resolves issue 1728.

1728. Type of an explicit instantiation of a variable template

Section: 17.7.2 [temp.explicit] **Status:** tentatively ready **Submitter:** Larisse Voufo **Date:** 2013-08-05

It is not clear to what extent the type in an explicit instantiation must match that of a variable template. For example:

```
template<typename T> T var = T();  
template float var<float>; // #1.  
template int* var<int>; // #2.  
template auto var<char>; // #3.
```

(See also issue 1704.)

Proposed resolution (May, 2017):

This issue is resolved by the resolution of issue 1704.

1836. Use of class type being defined in trailing-return-type

Section: _N4567_5.1.1 [expr.prim.general] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2014-01-17

According to _N4567_5.1.1 [expr.prim.general] paragraph 3,

Unlike the object expression in other contexts, **this* is not required to be of complete type for purposes of class member access (8.2.5 [expr.ref]) outside the member function body.

Is this special treatment of member access expressions intended to apply only to **this*, or does it apply to other ways of specifying the class being defined in the object expression? For example,

```
struct S {  
    int i;  
    auto f1() -> decltype((*this).i); // okay  
    auto f2(S& This) -> decltype(This.i); // okay?  
    auto f3() -> decltype(((S*)0)->i); // okay?  
};
```

There is implementation divergence on this question.

If the intent is to allow object expressions other than `*this` to have the current class type, this specification should be moved from `_N4567_.5.1.1 [expr.prim.general]` to 8.2.5 [expr.ref] paragraph 2, which is where the general requirement for complete object expression types is found.

On a related point, the note immediately following the above-cited passage is not quite correct:

[Note: only class members declared prior to the declaration are visible. —end note]

This does not apply when the member is a “member of an unknown specialization,” per 17.6.2.1 [temp.dep.type] paragraph 5 bullet 3 sub-bullet 1; for example,

```
template<typename T> struct S : T {
    auto f() -> decltype(this->x);
};
```

Here `x` is presumed to be a member of the dependent base `T` and is not “declared prior to the declaration” that refers to it.

Proposed resolution (May, 2017):

1. Change 8.1.2 [expr.prim.this] paragraph 2 as follows:

Unlike the object expression in other contexts, `*this` is not required to be of complete type for purposes of class member access (8.2.5 [expr.ref]) outside the member function body. [Note: Only class members declared prior to the declaration are visible. —end note] [Note: In a trailing-return-type, the class being defined is not required to be complete for purposes of class member access (8.2.5 [expr.ref]). Class members declared later are not visible. *Example:*

```
struct A {
    char g();
    template<class T> auto f(T t) -> decltype(t + g())
    { return t + g(); }
};
template auto A::f(int t) -> decltype(t + g());
```

—end example] —end note]

2. Change 8.2.5 [expr.ref] paragraph 2 as follows, splitting the paragraph into two paragraphs as indicated:

For the first option (dot) the first expression shall be a glvalue having complete class type. For the second option (arrow) the first expression shall be a prvalue having pointer to complete class type. In both cases, the class type shall be complete unless the class member access appears in the definition of that class. [Note: If the class is incomplete, lookup in the complete class type is required to refer to the same declaration (6.3.7 [basic.scope.class]). —end note]

The expression `E1->E2` is converted to the equivalent form `(* (E1)) . E2`; the remainder...

2273. Inheriting constructors vs implicit default constructor

Section: 15.1 [class.ctor] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-06-17

In an example like

```
struct A { A(int = 0); };
struct B : A { using A::A; };
B b0(0); // #1
B b;     // #2
```

Is #2 valid (presumably calling the constructor inherited from `A`, or ill-formed due to ambiguity with `B`'s implicit default constructor)?

Proposed resolution (May, 2017):

1. Change 10.3.3 [namespace.udecl] paragraph 16 as follows:

For the purpose of forming a set of candidates during overload resolution, the functions that are introduced by a *using-declaration* into a derived class are treated as though they were members of the derived class. In particular, the implicit `this` parameter shall be treated as if it were a pointer to the derived class rather than to the base class. This has no effect on the type of the function, and in all other respects the function remains a member of the base class. Likewise, constructors that are introduced by a *using-declaration* are treated as though they were constructors of the derived class when looking up the constructors of the derived class (6.4.3.1 [class.qual]) or forming a set of overload candidates (16.3.1.3 [over.match.ctor], 16.3.1.4 [over.match.copy], 16.3.1.7 [over.match.list]). If such a constructor is selected to perform the initialization of an object of class type, all subobjects other than the base class from which the constructor originated are implicitly initialized (15.6.3 [class.inhctor.init]). [Note: A member of a derived class is sometimes preferred to a member of a base class if they would otherwise be ambiguous (16.3.3 [over.match.best]). —end note]

2. Insert the following as a new bullet following 16.3.3 [over.match.best] bullet 1.7:

- ...
- `F1` and `F2` are function template specializations, and the function template for `F1` is more specialized than the template for `F2` according to the partial ordering rules described in 17.5.6.2 [temp.func.order], or, if not that,
- `F1` is a constructor for a class `D`, `F2` is a constructor for a base class `B` of `D`, and for all arguments the corresponding parameters of `F1` and `F2` have the same type. [Example:

```

struct A {
    A(int = 0);
};

struct B: A {
    using A::A;
    B();
};

int main() {
    B b; // OK, B::B()
}

```

—end example], or, if not that,

- F1 is generated from a *deduction-guide* (16.3.1.8 [over.match.class.deduct])...

This resolution also resolves issue 2277.

2277. Ambiguity inheriting constructors with default arguments

Section: 16.3.3.2 [over.ics.rank] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-06-23

In an example like:

```

struct A {
    A(int, int = 0);
    void f(int, int = 0);
};
struct B : A {
    B(int); using A::A;
    void f(int); using A::f;
}

```

calls to `B(int)` and `B::f(int)` are ambiguous, because they could equally call the version inherited from the base class. This doesn't match the intent in 10.3.3 [namespace.udecl], which usually makes derived-class functions take precedence over ones from a base class.

The above patterns are not common, although they sometimes cause breakage when refactoring a base class. However, P0136R1 brings this into sharp focus, because it causes the rejection of the following formerly-valid and very reasonable code:

```

struct A {
    A(int = 0);
};
struct B : A {
    using B::B;
};
B b;

```

Proposed resolution (May, 2017):

This issue is resolved by the resolution of issue 2273.

2287. Pointer-interconvertibility in non-standard-layout unions

Section: 6.9.2 [basic.compound] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-07-06

According to 12.3 [class.union] paragraph 2,

[Note: A union object and its non-static data members are pointer-interconvertible (6.9.2 [basic.compound], 8.2.9 [expr.static.cast]). As a consequence, all non-static data members of a union object have the same address. —end note]

However, the normative wording now only requires this for standard-layout unions.

Proposed resolution (April, 2017):

Change 6.9.2 [basic.compound] bullet 4.2 as follows:

Two objects *a* and *b* are *pointer-interconvertible* if:

- they are the same object, or
- one is a ~~standard-layout~~ union object and the other is a non-static data member of that object (12.3 [class.union]), or
- ...

2290. Unclear specification for overload resolution and deleted special member functions

Section: 16.3.1 [over.match.funcs] **Status:** tentatively ready **Submitter:** Howard Hinnant **Date:** 2016-07-21

According to 16.3.1 [over.match.funcs] paragraph 8,

A defaulted move constructor or assignment operator (15.8 [class.copy]) that is defined as deleted is excluded from the set of candidate functions in all contexts.

It is unclear whether this is intended to apply to all defaulted assignment operators or only move assignment operators.

Proposed resolution (April, 2017):

Change 16.3.1 [over.match.funcs] paragraph 8 as follows:

A defaulted move ~~constructor or assignment operator~~ **special function** (15.8 [class.copy]) that is defined as deleted is excluded from the set of candidate functions in all contexts.